

MÓDULO DE ANIMACIÓN · `src/js/modules/`

preHeaderScene.js

Explicación completa del código:
escena de cabecera con GSAP, ScrollTrigger
y efecto typewriter.

Narrativa · Soldados peruanos
Agosto 2026

1. Qué hace este archivo

preHeaderScene.js orquesta la cabecera previa al artículo (.v-n-preh). No es un scroll clásico de varias pantallas: fija (pin) el bloque entero y avanza una timeline de GSAP a medida que el usuario hace scroll. El progreso de esa timeline es el “guion” de tres fases visuales.

Las tres fases narrativas

- Fase 1 — Un rostro grande (el anónimo peruano), su atribución y la cita «Nos están mandando como carne de cañón», escrita con typewriter.
- Fase 2 — El mismo rostro se desplaza y encoge hasta ocupar su hueco en una rejilla de muchos rostros. El resto de caras aparece con un stagger aleatorio y entra un segundo texto (también typewriter).
- Fase 3 — La rejilla se apaga. El rostro principal viaja al hueco de la conversación (slot). Entra Pachas, la línea «Hermanos», el texto lead, la cita «¿Quieres irte a morir a la guerra?» y el cierre.

En móvil el orden de la fase 3 cambia: primero desaparece el rostro y se lee el lead; luego el rostro reaparece ya en el slot y sigue la conversación. En desktop el viaje al slot y la conversación ocurren a la vez, y el lead llega después.

Cómo se llama

main.js importa la función por defecto y la ejecuta en initAll(), al evento load de window. Si falta el DOM mínimo (.v-n-preh, escena y fase 1), la función sale sin hacer nada.

```
import preHeaderScene from "../modules/preHeaderScene";
preHeaderScene();
```

2. Dependencias (líneas 1–5)

```
import gsap from "gsap";
import { ScrollTrigger } from "gsap/ScrollTrigger";
import typewriter from "../helpers/typewriter";

gsap.registerPlugin(ScrollTrigger);
```

gsap Motor de animación. Timelines, tweens, getProperty, matchMedia, utils.clamp.

ScrollTrigger Plugin que ata el playhead de una timeline al scroll, y permite pin del bloque.

typewriter Helper del proyecto. Parte el texto con SplitText y revela caracteres (o palabras) en stagger. Devuelve { at, revert, ... }. at(n) es el instante (segundos) en el que aparece la unidad n (1-based).

registerPlugin es obligatorio una vez antes de usar ScrollTrigger. El typewriter registra SplitText por su cuenta.

3. Helpers de geometría e identidad

imgKey(el) — líneas 7–11

Dado un contenedor de cara, localiza el y extrae el nombre de archivo de src o currentSrc, sin ruta ni query string. Ejemplo: assets/images/mockup-face1.webp → mockup-face1.webp.

Sirve para emparejar el rostro grande de la fase 1 con el hueco de la fase 2 que usa la misma foto. No se usa un id en el HTML: la identidad es el fichero.

```
function imgKey(el) {
  const img = el.querySelector("img");
  const src = img?.getAttribute("src") || img?.currentSrc || "";
  return src.split("/").pop().split("?")[0];
}
```

- Optional chaining (?.) evita errores si no hay img.
- split("?")[0] ignora cache-busters tipo ?v=123.
- Si no hay src, devuelve cadena vacía.

pickHomeFace(mainFace, faces) — líneas 13–34

Elige, entre las caras de la fase 2, cuál es el “hogar” del rostro principal: el hueco al que debe viajar en el morph de fase 1 → 2.

Algoritmo:

- 1. Calcula la clave de imagen de mainFace.
- 2. Filtra faces que tienen la misma clave. Si no hay ninguna, null.
- 3. Si hay varias (el mockup usa mockup-face1.webp muchas veces), elige la más cercana en pantalla al centro de mainFace, con distancia euclídea al cuadrado (no hace falta la raíz: el orden se conserva).

getBoundingClientRect() da coordenadas de viewport. Se usa el centro (left + width/2, top + height/2). El resultado se guarda en homeFace; el resto de caras es otherFaces (las que harán fade-in).

destTo(el, target) — líneas 36–48

Calcula el x, y y scale que hay que aplicar a el para que su caja visual coincida con la de target. Es el truco del “FLIP” a mano: el DOM no se mueve de padre; se transforma el mismo nodo hasta cubrir el hueco destino.

```
const x = Number(gsap.getProperty(el, "x")) || 0;
const y = Number(gsap.getProperty(el, "y")) || 0;
const scale = Number(gsap.getProperty(el, "scale")) || 1;
const from = el.getBoundingClientRect();
const to = target.getBoundingClientRect();
return {
  x: to.left - (from.left - x),
  y: to.top - (from.top - y),
  scale: to.width / (from.width / scale),
};
```

Por qué resta (from.left - x)

getBoundingClientRect ya incluye el translate actual. Si el elemento ya tiene x = 40, from.left está 40 px más a la derecha que su posición “lógica”. from.left - x recupera la esquina sin transform. Luego to.left - esa esquina es el translate que GSAP debe poner para alinear izquierdas.

La escala

from.width / scale es el ancho “sin escala”. to.width dividido por eso da el factor para que el ancho visual coincida con el destino. Se asume que el aspect ratio es el mismo (fotos de cara). transformOrigin se fija luego en 0% 0% para que

x/y y scale compartan la esquina superior izquierda, coherente con left/top del rect.

Los destinos se guardan en objetos mutables dest2 (fase 2 / homeFace) y dest3 (fase 3 / slotFace). Las tweens leen dest2.x con funciones () => dest2.x para que, al hacer refresh (resize), ScrollTrigger vuelva a interpolar con valores nuevos (invalidateOnRefresh).

4. Offset del chrome (header + banner)

headerPinOffset() — líneas 50–60

El pin no puede empezar en top: 0 porque el header del site (y a veces un banner .v-h-a) ocupan la franja superior. Esta función devuelve los píxeles que hay que respetar:

- Desktop (≥ 699 px): header 58 px. Móvil: 50 px.
- Si existe .v-h-a, está display distinto de none y tiene offsetHeight > 0, suma 48 px más.

Esas cifras están hardcodeadas (no se miden del DOM del header). Si el header real cambia de altura, hay que actualizarlas aquí y en CSS si aplica.

syncChromeOffset() — líneas 62–66

Escribe el mismo número en la custom property --preh-offset del <html>. El CSS de la escena puede usar esa variable (por ejemplo para height o top). Devuelve el offset para quien lo necesite. Se llama al iniciar y en cada onRefresh de ScrollTrigger.

5. Arranque de preHeaderScene()

Captura de nodos — líneas 68–98

Primero resuelve el árbol: preh → scene → phase1/2/3. Si faltan preh, scene o phase1, return inmediato: no hay escena que animar.

Referencias clave:

.v-n-preh__face--main Rostro grande de fase 1. Es el único que se transforma (x/y/scale).

.v-n-preh__txt-slash (fase 1) Trazo SVG sobre la cara; aparece tras las primeras letras.

.v-n-preh__sub-c / __sub Atribución «Anónimo» / «Peruano».

.v-n-preh__txt (fase 1) Cita inicial. Typewriter.

.v-n-preh__t (fase 2) Texto de la rejilla. Typewriter.

.v-n-preh__faces / __face Las ~21 caras de la rejilla.

homeFace Cara de la rejilla emparejada con mainFace (pickHomeFace).

otherFaces El resto; fade-in en fase 2, fade-out al pasar a fase 3.

slotFace Hueco destino en fase 3 (Anónimo). Selectores: --slot o --1, por si el HTML varía.

pachasFace Cara de Pachas (--pachas o --2).

pachasSub / anonSub Etiquetas bajo cada cara; se buscan en el padre.

hermanos / convoLine Texto «Hermanos» y la raya que se escala en X.

txtLead / txtQuote / txtEnd Párrafos de fase 3.

quoteSlash SVG slash sobre la cara de Pachas, sincronizado con la cita.

Estado inicial GSAP — líneas 100–108

- dest2 y dest3 empiezan en { x:0, y:0, scale:1 } hasta el primer syncDest.
- Todas las faces de fase 2, homeFace y slotFace: autoAlpha 0 (opacity + visibility).
- convoLine: scaleX 0, origen left center → la raya crece de izquierda a derecha.
- syncChromeOffset() deja --preh-offset listo antes de medir nada de pin.

homeFace se oculta porque el “viaje” lo hace mainFace encima; no queremos dos copias visibles. slotFace también se oculta: el hueco de Anónimo en fase 3 lo cubre mainFace transformado, no la img del slot.

6. Timeline de intro (no depende del scroll)

líneas 110–125. Se reproduce sola al cargar, con duration 0.7 y ease power2.out por defecto. No tiene ScrollTrigger.

- mainFace: fromTo autoAlpha 0 → 1.
- subsPhase1 (Anónimo, Peruano): fromTo autoAlpha 0 → 1, stagger 0.12, posición "<" (empiezan a la vez que el tween anterior).
- typewriter(txtPhase1, { timeline: intro, position: "-=0.5" }) solapa el mecanografiado 0.5 s antes de que acabe lo anterior.
- El slash de fase 1 entra en ``<${typedPhase1?.at(5) ?? 0}``: posición relativa al tween anterior, desplazada al instante en que aparece el 5.º carácter. Si typewriter falló (null), el offset es 0.

El intro NO espera al usuario. La escena “en vivo” (fases 2 y 3) solo se construye cuando intro termina: `intro.then(() => { ... })`.

Eso evita que SplitText / destinos se calculen mientras el texto o las imágenes aún están animando la entrada, y da un momento de lectura de la cita.

7. Helpers internos de la escena

syncDest() — líneas 127–130

Vuelca destTo(mainFace, homeFace) en dest2 y destTo(mainFace, slotFace) en dest3. Object.assign muta los mismos objetos que leen las tweens. Llamar esto en resize/refresh es lo que mantiene el morph alineado si cambia el layout.

addLead(scroll, position) — líneas 132–149

Añade el párrafo lead de fase 3 a la timeline de scroll:

- fromTo autoAlpha 0 → 1, 0.55 s, power1.out, immediateRender: false.
- immediateRender: false es importante en un timeline con scrub: si no, GSAP aplicaría el “from” al crear el tween y el texto quedaría invisible demasiado pronto.
- Luego typewriter con fill: true y fillFrom: 0.3: las letras ya se ven al 30 % y se “rellenan” a 1. stagger.amount 2.6

reparte ese relleno a lo largo de 2.6 s (el stagger total no es 0.03 por letra). duration 0.02 por carácter. showTarget: false porque la visibilidad del contenedor la controla el fromTo.

- El typewriter se coloca en `\${position}+=0.55` (justo después del fade-in).

addConvo(scroll, start) — líneas 151–177

Coreografía de la “conversación” a partir de un instante start (label):

- anonSub → autoAlpha 1 a start+0.15 (0.3 s).
- convoline → scaleX 1 a start+0.35 (0.5 s): la raya se dibuja.
- hermanos → autoAlpha 1 a start+0.45.
- pachasFace + pachasSub juntos → autoAlpha 1 a start+0.85 (0.45 s). filter(Boolean) por si falta alguno en el HTML.

No anima slotFace: ese hueco lo cubre mainFace.

addQuote(scroll, position) — líneas 179–197

Hace visible txtQuote con scroll.set (autoAlpha 1) y lanza typewriter corto (stagger.amount 0.95). El slash de Pachas hace fade-in cuando llega el 5.º carácter de la cita (typedQuote.at(5)). Devuelve el objeto typed para remember().

addEnd(scroll, position) — líneas 199–208

Igual patrón: set autoAlpha 1 + typewriter del cierre, stagger.amount 1.15. showTarget: false.

8. matchMedia y la timeline de scroll

Todo lo que sigue vive dentro de intro.then. gsap.matchMedia() crea contextos que se destruyen y reconstruyen al cruzar el breakpoint 699 px (el mismo umbral que headerPinOffset).

```
mm.add({
  isMobile: "(max-width: 698px)",
  isDesktop: "(min-width: 699px)",
}, (context) => { ... });
```

Solo una condición es true a la vez. El callback recibe context.conditions. El return del callback es la función de cleanup: aquí recorre reversiones y llama typed.revert() para deshacer SplitText cuando matchMedia reconstruye (si no, el DOM se llena de spans huérfanos).

remember / reversiones

Cada typewriter metido en la timeline de scroll se registra. Al cambiar de breakpoint, GSAP mata las tweens del contexto, pero SplitText hay que revertirlo a mano.

Timeline scroll — líneas 227–230

paused: true porque no la reproduce el reloj: la mueve ScrollTrigger con scrub. defaults ease: none → el scrub 1:1 se siente lineal respecto al scroll. Los tweens individuales pueden pisar el ease (power2.out, power2.inOut...).

9. Guion común (fase 1 → 2)

Ocurre igual en móvil y desktop, antes del label phase3.

1. Apagar copy de fase 1

fromTo opacity 1 → 0 de [txtPhase1, subCPhase1, txtDashPhase1], 0.65 s, power2.out, immediateRender: false. Usa opacity, no autoAlpha: el intro ya los dejó visibles; aquí solo se desvanecen.

2. Texto de fase 2

En el segundo 0.65: set autoAlpha 1 en txtPhase2 y typewriter con amount 1.25, showTarget: false. remember() guarda el revert.

3. Resto de caras

Si hay otherFaces, to autoAlpha 1, 0.5 s, stagger amount 1 from: "random", posición 0.7. No entran en orden de DOM: el azar refuerza la idea de “muchos”.

4. Morph de mainFace → homeFace

Si existen ambos, tween a dest2 (x, y, scale) con transformOrigin 0% 0%, 0.9 s, power2.inOut, posición 0.55. Empieza un poco antes que el fade de las otras caras, para que el viaje se lea como causa de la rejilla.

Las propiedades son funciones: x: () => dest2.x. Con invalidateOnRefresh, GSAP reevalúa esos getters tras syncDest().

5. Label phase3 y fade-out de fase 2

fadeOutP2 = txtPhase2 + otherFaces. Se desvanecen en el label "phase3" (0.55 s). homeFace no está en esa lista (ya estaba en autoAlpha 0). mainFace sigue visible: es el puente a la fase 3.

10. Rama móvil vs desktop (fase 3)

Móvil (≤ 698 px) — líneas 295–326

Secuencia pensada para poco alto de pantalla: no caben lead y conversación a la vez.

- En phase3: mainFace autoAlpha 0 (0.4 s). El rostro se va con la rejilla.
- Label lead = phase3+0.4. addLead: el texto largo ocupa el centro.
- Label phase3b = +=0.25 (0.25 s después de lo último). El lead se apaga (0.4 s).
- En phase3b: set de mainFace a dest3 (salto, no interpolación) y luego autoAlpha 1 a phase3b+0.15. En móvil no se ve el vuelo al slot: aparece ya colocado, porque el lead ha tapado el viaje.
- addConvo(scroll, "phase3b").
- Label quote = phase3b+1.4 → addQuote.
- Label endCopy = +=0.12 → addEnd.

Desktop (≥ 699 px) — líneas 327–349

Hay sitio: el morph y la conversación conviven; el lead llega después.

- En phase3: tween de mainFace a dest3, 0.75 s, power2.inOut (sí se ve el viaje desde la rejilla hasta el slot de Anónimo).
- addConvo(scroll, "phase3") en paralelo al morph.

- Label lead = phase3+1.7 → addLead (después de que la convo haya asentado).
- quote = +=0.15, endCopy = +=0.12.

Cola de lectura — línea 352

```
scroll.to({}, { duration: 1.4 });
```

Tween vacío: no anima nada, solo alarga la timeline 1.4 s. Con scrub, eso se traduce en más píxeles de scroll al final para que el usuario pueda leer el cierre sin que el pin se suelte enseguida.

11. ScrollTrigger (líneas 354–373)

```
ScrollTrigger.create({
  animation: scroll,
  trigger: preh,
  start: () => `top ${headerPinOffset()}px`,
  end: () => { ... },
  pin: true,
  anticipatePin: 1,
  scrub: true,
  invalidateOnRefresh: true,
  onRefresh: refreshScene,
});
```

animation: scroll La timeline paused se engrana al scroll. progress 0 = inicio del pin, 1 = fin.

trigger: preh El bloque .v-n-preh. pin: true fija ESE elemento, no la ventana.

start Cuando el top de preh llega a headerPinOffset() px del viewport. Función para recalcular si aparece/desaparece el banner.

end Distancia de scroll del pin. Ver fórmula abajo.

pin: true El bloque queda fijo; el scroll “consume” la duración de la timeline.

anticipatePin: 1 Aplica el pin un frame antes para reducir el salto (jump) al fijar.

scrub: true El playhead sigue el scroll al instante (sin lag). Hacia atrás revierte la escena.

invalidateOnRefresh Al resize/refresh, GSAP invalida valores de las tweens y relea los getters (dest2/dest3).

onRefresh: refreshScene syncChromeOffset + syncDest. Primero CSS offset, luego medir rects.

Fórmula del end

```
const vh = window.innerHeight;
const scaled = scroll.duration() * vh * 1.1;
return `+=${Math.round(gsap.utils.clamp(vh * 5.5, vh * 9.5, scaled))}`;
```

scroll.duration() es la duración en segundos de la timeline (en GSAP, con scrub, 1 segundo de timeline no es 1 segundo de reloj: es una unidad de progreso). Se multiplica por la altura de ventana y por 1.1 para convertir esa duración en píxeles de scroll. Luego se clampa entre 5.5 y 9.5 viewports:

- Mínimo 5.5 vh: aunque la timeline sea corta, hay recorrido de lectura.
- Máximo 9.5 vh: evita un pin interminable si se añaden muchos typewriters.

El string `+=N` significa “N píxeles después del start”. El pin dura exactamente eso. pinSpacing (por defecto true) inserta un spacer para que el resto de la página no salte debajo.

12. Mapa de tiempos (aproximado)

Los números son posiciones en la timeline de scroll (segundos de GSAP), no tiempo de reloj. Con scrub, “0.65” es un porcentaje del pin. Los typewriters alargan la duración real según el texto.

Tramo compartido

- 0.00 — empieza el fade-out de cita y atribución de fase 1.
- 0.55 — mainFace inicia el morph hacia homeFace (0.9 s).
- 0.65 — cita fase 1 ya casi fuera; typewriter de txtPhase2.
- 0.70 — otherFaces aparecen (stagger aleatorio, 0.5 s).
- phase3 — fade-out de texto fase 2 y otherFaces.

Móvil desde phase3

- phase3 — mainFace se oculta.
- lead (phase3+0.4) — lead + typewriter fill.
- phase3b (+0.25) — lead out; mainFace salta a dest3 y reaparece; convo.
- quote (phase3b+1.4) — cita de Pachas.
- endCopy (+0.12) — cierre. + 1.4 s de cola.

Desktop desde phase3

- phase3 — morph a dest3 (0.75 s) + convo en paralelo.
- lead (phase3+1.7) — párrafo lead.
- quote (+0.15) y endCopy (+0.12). + 1.4 s de cola.

13. Decisiones de diseño (por qué está escrito así)

Un solo nodo viaja

mainFace es el único elemento que se mueve con x/y/scale. homeFace y slotFace son “imanes” invisibles. Así no hay dos fotos del anónimo a la vez y el ojo sigue un solo sujeto de principio a fin.

Identidad por fichero, no por clase

pickHomeFace usa el src. Sirve con el mockup (face1 repetido) eligiendo el hueco más cercano. En producción, si cada soldado tiene foto única, el filtro por clave basta y la distancia es un desempate.

Intro separado del scroll

La primera cita se lee sin exigir scroll. El pin empieza cuando el usuario decide bajar. intro.then evita medir destinos con opacidades a medias.

matchMedia en vez de un if al inicio

Si el usuario gira el móvil o ensancha la ventana, GSAP destruye el contexto, revierte SplitText y monta la otra rama. Un `if` suelto al `load` no se actualizaría.

Getters en x/y/scale

Sin `() => dest2.x`, GSAP cachearía el número del primer frame (a menudo 0,0,1) y el `morph` fallaría tras el primer layout o al refrescar.

fill en el lead

El lead es un párrafo largo. Un typewriter desde `opacity 0` letra a letra se haría eterno y difícil de leer con `scrub`. `fillFrom 0.3` deja el texto presente y el `stagger` solo “enciende” el contraste.

14. Relación con HTML y CSS

El módulo asume la estructura de `pre-article-header.hbs`:

- `.v-n-preh > .v-n-preh__scene > tres .v-n-preh__phase--1/2/3`
- Fase 2: `.v-n-preh__faces` con muchos `.v-n-preh__face` e `imgs`.
- Fase 3: `.v-n-preh__face--slot` (Anónimo), `--pachas`, `convo-dash`, textos `lead/quote/end`.

Las tres fases están en el mismo `scene`, probablemente superpuestas en CSS (posición absoluta). JS no cambia `display` entre fases: solo opacidades y el `transform` de `mainFace`. `--preh-offset` sincroniza el pin con el `chrome`.

El indicador de scroll (`.scr-ind`) no lo anima este módulo: `main.js` llama `fadeOnScroll(".v-n-preh .scr-ind")`.

15. Glosario GSAP usado aquí

autoAlpha Combina `opacity` y `visibility`. `0` → `hidden`; `>0` → `visible`.

fromTo Estado inicial y final explícitos (más seguro que `from` o `to` solos).

set Aplica valores en un instante, sin interpolar (salto).

stagger Retraso entre elementos. `amount`: reparte el total entre todos.

`"<"` Position parameter: alinea este tween con el inicio del anterior.

`"-=0.5"` Solapa 0.5 s con el final del anterior.

`"+=0.25"` Empieza 0.25 s después de lo último de la timeline.

`"phase3+=0.4"` 0.4 s después del label `phase3`.

addLabel Marca un nombre en el tiempo para anclar otros tweens.

matchMedia Contextos GSAP ligados a media queries, con `cleanup`.

scrub `progress` de la animación = `progress` del scroll entre `start` y `end`.

pin `position: fixed` (o `transform`) del trigger mientras el pin está activo.

gsap.getProperty Lee `x`, `y`, `scale` actuales aunque estén en el `transform matrix`.

gsap.utils.clamp(min, max, val) Acota `val` al intervalo `[min, max]`.

16. Recorrido lineal del fichero

Líneas	Qué ocurre
1–5	Imports y registro de ScrollTrigger.
7–11	imgKey: nombre de archivo de la foto.
13–34	pickHomeFace: hueco de fase 2 más cercano con la misma foto.
36–48	destTo: translate+scale para cubrir un target.
50–66	Offset header/banner y CSS --preh-offset.
68–74	Early return si no hay DOM de escena.
76–98	Query de todos los actores (caras, textos, slash, convo).
100–108	dest2/dest3 y estados iniciales (caras y raya ocultas).
110–125	Intro: cara, atribución, typewriter, slash. Sin scroll.
127–130	syncDest alinea dest2 y dest3 con el layout actual.
132–208	addLead, addConvo, addQuote, addEnd: ladrillos de fase 3.
210–218	Cuando acaba el intro: matchMedia 698/699.
221–223	remember() apila reverts de SplitText.
227–282	Timeline scroll: apaga fase 1, typewriter 2, caras, morph.
284–293	Label phase3 y fade de la rejilla.
295–326	Fase 3 móvil: lead → salto al slot → convo → quote → end.
327–349	Fase 3 desktop: morph al slot + convo → lead → quote → end.
352	Cola vacía de 1.4 s para leer.
354–373	ScrollTrigger: pin, scrub, end clampado, refresh destinos.
375–377	Cleanup de matchMedia: revert typewriters.

17. Cosas que este archivo no hace

- No carga imágenes ni espera decode: asume que load de window ya ocurrió.
- No implementa prefers-reduced-motion (sí el helper typewriter).
- No anima el scroll-indicator.
- No mata el matchMedia al salir de la página (no es SPA).
- No usa Flip plugin: destTo es un Flip manual de un solo elemento.
- Las alturas 50/58/48 del chrome están escritas a mano.

18. Cómo leerlo la próxima vez

Empieza por el HTML de las tres fases para tener los nombres. Luego lee destTo y pickHomeFace: son el motor espacial. Después el intro. Por último la timeline scroll de arriba abajo, saltando addLead/addConvo como “macros”. El ScrollTrigger del final solo dice: “esta timeline dura N viewports de scroll y el bloque no se mueve”.

Si algo “no llega al hueco”, el sospechoso es syncDest (rects mal medidos, imágenes aún sin tamaño, o transformOrigin distinto de 0% 0%). Si el texto se parte mal al rotar el móvil, el sospechoso es remember/revert de SplitText.